

```
In [1]: import pandas as pd
        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import StandardScaler
        from sklearn.linear_model import LogisticRegression
        from sklearn.metrics import confusion_matrix, accuracy_score
```

```
In [2]: df=pd.read_csv(r"C:\Users\Dell\Social_Network_Ads.csv")
        df
```

```
Out[2]:
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0
...	...	...	...
395	46	41000	1
396	51	23000	1
397	50	20000	1
398	36	33000	0
399	49	36000	1

400 rows × 3 columns

```
In [3]: df.isnull().sum()
```

```
Out[3]: Age          0
        EstimatedSalary  0
        Purchased      0
        dtype: int64
```

```
In [4]: x=df.drop(['Purchased'],axis=1)
        x
```

Out[4]:

	Age	EstimatedSalary
0	19	19000
1	35	20000
2	26	43000
3	27	57000
4	19	76000
...	...	...
395	46	41000
396	51	23000
397	50	20000
398	36	33000
399	49	36000

400 rows × 2 columns

```
In [5]: y=df['Purchased']  
y
```

```
Out[5]: 0      0  
1      0  
2      0  
3      0  
4      0  
..  
395    1  
396    1  
397    1  
398    0  
399    1  
Name: Purchased, Length: 400, dtype: int64
```

```
In [7]: x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25)
```

```
In [8]: sc=StandardScaler()  
x_train=sc.fit_transform(x_train)  
x_test=sc.transform(x_test)
```

```
In [9]: classifier=LogisticRegression()  
classifier.fit(x_train,y_train)
```

```
Out[9]: ▼ LogisticRegression ⓘ ?  
        ► Parameters
```

```
In [10]: y_test_pred=classifier.predict(x_test)
```

```
In [13]: x_test
```

```
Out[13]: array([[ -0.92863358,  0.31770514],
 [ -1.49757179, -0.15501174],
 [ -0.45451841, -0.50954941],
 [  0.87300407, -1.33680395],
 [  2.01088048,  0.22907072],
 [  0.11441979,  0.81996683],
 [ -0.73898751,  1.14495968],
 [  2.01088048,  0.43588436],
 [  0.3988889 ,  1.05632527],
 [ -1.02345662,  0.58360838],
 [  1.34711924,  1.35177332],
 [ -0.45451841, -0.74590785],
 [  0.30406586,  0.1108915 ],
 [ -1.30792572, -0.39137019],
 [  0.11441979,  1.58813176],
 [  0.87300407, -1.27771434],
 [  0.11441979, -0.77545265],
 [ -0.26487234,  0.14043631],
 [ -0.73898751,  0.61315319],
 [ -1.02345662,  0.61315319],
 [  2.10570351, -0.77545265],
 [  0.11441979,  1.11541488],
 [ -1.11827965,  0.34724994],
 [  2.01088048, -1.15953512],
 [  0.30406586,  0.31770514],
 [  1.15747317, -1.42543837],
 [ -0.26487234, -0.27319097],
 [ -0.45451841, -1.10044551],
 [ -1.02345662, -0.30273577],
 [  0.58853497,  2.09039345],
 [ -0.26487234, -1.42543837],
 [  0.9678271 ,  2.14948306],
 [ -0.92863358, -0.39137019],
 [  0.20924283, -0.09592213],
 [ -0.07522627,  2.23811747],
 [ -0.26487234,  0.6722428 ],
 [ -1.87686392, -0.00728772],
 [  0.30406586, -0.68681824],
 [ -0.07522627,  0.34724994],
 [  0.3988889 ,  0.14043631],
 [ -1.21310268, -1.0413559 ],
 [ -0.54934144,  1.44040773],
 [  0.9678271 ,  1.94266942],
 [ -1.02345662,  0.64269799],
 [  0.3988889 ,  0.64269799],
 [  0.30406586, -0.4800046 ],
 [ -0.83381055,  2.35629669],
 [ -1.11827965,  0.34724994],
 [  0.20924283,  0.28816033],
 [ -0.64416448,  1.46995254],
 [  0.3988889 ,  0.05180189],
 [  2.10570351, -0.65727343],
 [ -1.21310268,  0.64269799],
 [ -1.49757179, -0.39137019],
 [ -0.64416448, -0.00728772],
 [  1.15747317, -0.95272148],
 [  0.87300407,  1.14495968],
 [ -0.07522627,  2.03130384],
 [  0.9678271 ,  0.16998111],
 [ -1.11827965, -1.57316239],
```

```

[-0.73898751,  0.34724994],
[-1.87686392,  0.40633955],
[-1.11827965, -1.51407278],
[-1.02345662,  0.46542916],
[-0.17004931,  1.67676618],
[-0.07522627,  0.16998111],
[-0.92863358, -0.27319097],
[-0.73898751, -1.51407278],
[ 2.01088048,  1.8244902 ],
[-0.64416448, -0.06637733],
[-1.11827965, -1.57316239],
[ 1.34711924,  2.06084864],
[ 2.10570351,  1.17450449],
[ 0.77818103, -1.18907993],
[ 0.683358   , -1.36634876],
[-0.26487234, -0.30273577],
[ 0.87300407, -0.74590785],
[-1.11827965,  0.52451877],
[-0.64416448,  0.22907072],
[-0.45451841, -1.18907993],
[ 0.20924283,  0.19952592],
[ 0.683358   , -1.24816954],
[ 0.01959676, -0.53909421],
[ 0.87300407,  1.32222851],
[-1.11827965, -0.74590785],
[-0.26487234,  0.25861553],
[ 0.11441979,  0.19952592],
[-0.17004931,  1.46995254],
[-0.07522627,  2.29720708],
[-1.68721785,  0.40633955],
[-0.26487234,  0.19952592],
[ 0.01959676,  0.0813467 ],
[ 0.58853497, -0.86408707],
[-0.73898751,  0.31770514],
[ 1.15747317,  0.58360838],
[ 1.06265014,  0.16998111],
[-1.68721785, -0.56863902],
[ 0.30406586, -0.15501174],
[ 0.49371193,  1.7949454 ],
[-0.64416448, -0.30273577]])

```

In [14]: `y_test_pred`

Out[14]: `array([0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0])`

In [15]: `classifier.predict([[ -0.92863358, 0.31770514]])`

Out[15]: `array([0])`

In [17]: `classifier.predict([[2.01088048, 0.22907072]])`

Out[17]: `array([1])`

In [19]: `matrix=confusion_matrix(y_test,y_test_pred)`
`matrix`

```
Out[19]: array([[58,  5],
                [12, 25]])
```

```
In [24]: score=accuracy_score(y_test,y_test_pred)
score
```

```
Out[24]: 0.83
```

```
In [22]: from sklearn.metrics import classification_report
print(classification_report(y_test,y_test_pred))
```

	precision	recall	f1-score	support
0	0.83	0.92	0.87	63
1	0.83	0.68	0.75	37
accuracy			0.83	100
macro avg	0.83	0.80	0.81	100
weighted avg	0.83	0.83	0.83	100

```
In [25]: print('True Positive:', matrix[0][0])
print('True Negative:', matrix[1][1])
print('False Positive:', matrix[0][1])
print('False Negative:', matrix[1][0])
print('Accuracy:', score)
print('Error Rate:', 1-score)
precision = matrix[0][0]/(matrix[0][0]+matrix[0][1])
print('Precision:', precision)
```

```
True Positive: 58
True Negative: 25
False Positive: 5
False Negative: 12
Accuracy: 0.83
Error Rate: 0.17000000000000004
Precision: 0.9206349206349206
```

```
In [ ]:
```